CLOUD PROJECT PRESENTATION

AI-Driven CI/CD Pipeline Failure Root Cause Analysis & Auto-Remediation System

February 2026

SRM Institute of Science and Technology

Project Information

Team & Overview

Team Members

AB
Arnavi Barve
RA2311033010026

VS
Vineet Sahoo
RA2311033010053

Faculty Guide

Mrs. Parimala G

Assistant Professor

Course Details

21IPE315P – Cloud Product and Platform Engineering

Department of Computational Intelligence

College of Engineering and Technology

The CI/CD Challenge

Modern software development relies on automated pipelines—but failures are inevitable

CI/CD Pipeline Essentials

Continuous Integration and Continuous Deployment pipelines automate code integration, build processes, testing, containerization, and deployment—reducing manual intervention and improving release consistency.

Common Failure Causes

Pipelines frequently encounter failures from configuration errors, dependency mismatches, infrastructure constraints, and resource limitations—disrupting deployment workflows and extending troubleshooting time.

Our Solution

An AI-powered system that integrates automated observability, failure classification, and AI-assisted reasoning to identify failure causes and provide actionable remediation guidance—reducing MTTR and manual debugging overhead.

The Problem

Manual

troubleshooting is time-consuming and inefficient in complex distributed environments

Pipeline Stages

1. Code Integration
2. Build Process
3. Testing
4. Containerization
5. Deployment

02. Real-Time Scenario

Cloud-Based Deployment System

A practical DevOps environment on AWS with automated CI/CD workflows

System Workflow Architecture

GitHub — Source Control → **Jenkins** — CI/CD Pipeline → **Docker** — Containerization → **Amazon ECR** — Image Registry → **Amazon EKS** — Kubernetes

AWS Platform

Cloud-based internal application deployment system leveraging AWS managed services for scalability and reliability.

Amazon EKS | Amazon ECR

Failure Points

Failures may arise at build, test, or deployment stages due to misconfigured manifests, dependency conflicts, or resource limitations.

Build Errors | Test Failures

AI-Powered Analysis

Pipeline logs, runtime errors, and Kubernetes events are automatically collected via CloudWatch and Prometheus, processed using AWS Lambda and analyzed through Amazon Bedrock.

CloudWatch | Bedrock AI

03. Challenges Identified

Key Obstacles in CI/CD Pipelines

Four critical challenges that disrupt deployment workflows and increase recovery time

01 Configuration Errors

CI/CD pipelines frequently fail due to incorrect Kubernetes configurations, including improper resource limits, misconfigured environment variables, or syntax errors in deployment manifests.

⚠ Impact: Deployment rejection, pod instability, repeated pipeline failures

02 Dependency Mismatches

Build and runtime failures commonly arise from missing dependencies, incompatible library versions, or unverified third-party updates. Identifying the specific dependency requires manual log analysis.

⚠ Impact: Build interruptions, delayed deployments, increased debugging effort

03 Infrastructure Constraints

Insufficient CPU, memory, or storage allocation can lead to container crashes and unsuccessful deployments. Kubernetes pods may terminate with OOMKilled errors when memory limits are incorrectly defined.

⚠ Impact: Runtime instability, service downtime, extended recovery time

04 Manual Troubleshooting Overhead

Traditional DevOps workflows rely heavily on manual inspection of CI/CD logs, Kubernetes events, and system metrics. This process is repetitive, time-consuming, and prone to human error in distributed systems.

⚠ Impact: Increased MTTR, reduced developer productivity

04. Proposed Solution

AI-Powered Failure Analysis System

Automated detection, analysis, and remediation for CI/CD pipeline failures

Intelligent DevOps Automation

The system automatically detects, analyzes, and assists in remediating failures—minimizing manual troubleshooting, improving pipeline reliability, and reducing Mean Time to Recovery (MTTR).

1 Data Collection

CloudWatch & Prometheus gather logs, events, and metrics

2 Log Processing

AWS Lambda parses and structures failure data

3 Classification

Categorize failures: build, test, config, dependency, infra

4 AI Analysis

Amazon Bedrock generates root cause explanations

5 Remediation

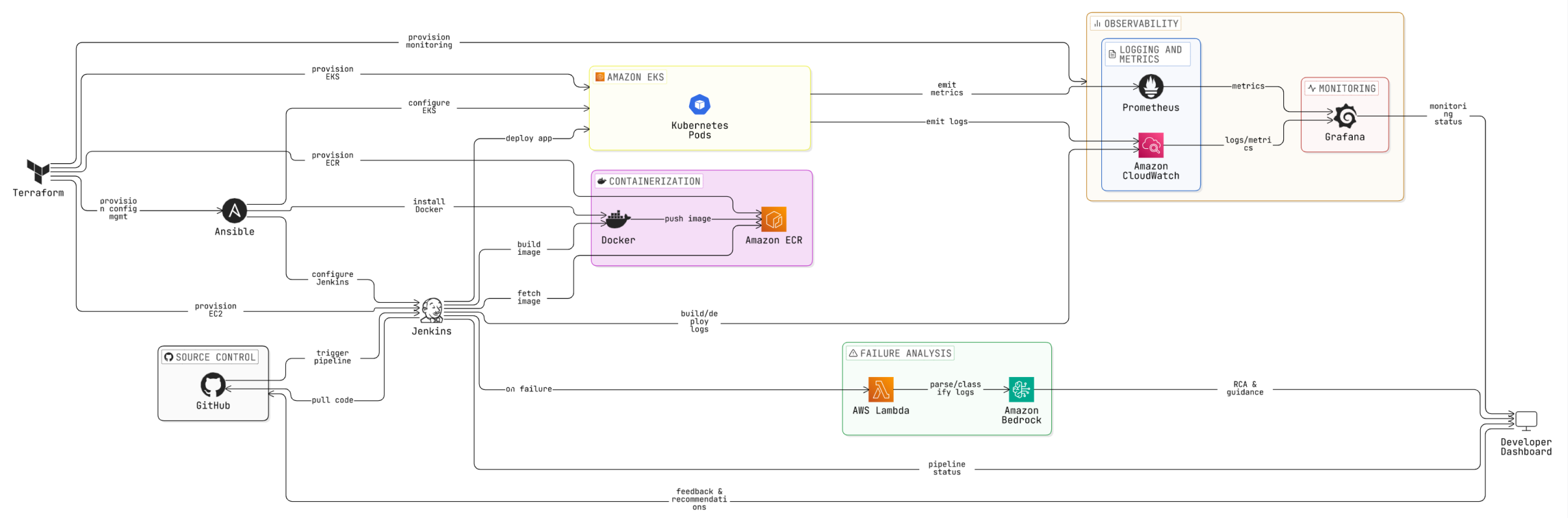
Controlled approval & guided corrective actions

- Automated Observability
- AI-Assisted Reasoning
- Controlled Approval
- Reduced MTTR

05. System Architecture

Integrated DevOps Automation Framework

Layered architecture ensuring separation of responsibilities, scalability, and operational clarity



Automated Build, Test & Deployment

Consistent and repeatable pipeline orchestrated by Jenkins

Pipeline Execution Flow

Git Commit
Trigger Pipeline

→

Build Stage
Compile App

→

Test Stage
Unit & Integration

→

Containerize
Docker Image

→

Push to ECR
Image Registry

→

Deploy
EKS Cluster

Pipeline Trigger

Developer commits source code or configuration to GitHub repository, automatically triggering the Jenkins CI/CD pipeline execution.

Quality Assurance

Automated testing phases including unit and integration tests validate functional correctness and system stability before deployment.

Failure Analysis

On pipeline failure, Jenkins forwards logs to AWS Lambda and Amazon Bedrock for classification, root cause analysis, and remediation guidance.

Practical Implementation Assessment

Evaluating technical, economic, and operational viability

Technical

Built using established, widely adopted technologies with high availability and seamless interoperability.

- AWS Managed Services
- Jenkins CI/CD
- Docker & Kubernetes
- Terraform & Ansible

Proven Technologies

Economic

Usage-based AWS pricing model enables cost control for academic and prototype-scale implementations.

Cost Components
Pipeline executions, compute, storage

Monitoring Services
CloudWatch, Prometheus, Grafana

AI Inference
Amazon Bedrock requests

Cost-Effective

Operational

Compatible with standard development practices and familiar tools, minimizing training overhead.

- Familiar GitHub Workflow
- Jenkins Integration
- Approval-Based Remediation
- Background Automation

Developer-Friendly

09. Expected Outcomes

Projected Benefits & Improvements

Measurable impact on software delivery workflows

Reduced MTTR

Mean Time to Recovery

Automated failure detection, log analysis, and root cause identification enable rapid diagnosis and structured remediation—significantly reducing recovery time.

Fewer Recurring Failures

AI-assisted reasoning provides precise corrective recommendations, preventing repeated pipeline failures and improving deployment stability.

- Config Corrections
- Dependency Updates
- Resource Adjustments

Enhanced Deployment Reliability

Consistent build validation, automated testing, containerized deployment, and continuous monitoring ensure only verified application artifacts are released.

- Build Validation
- Automated Testing

Improved Developer Productivity

Centralized dashboard provides visibility into pipeline status, failure summaries, and remediation guidance—enabling informed decision-making.

- Centralized Dashboard
- Real-time Visibility
- Guided Remediation

CONCLUSION

Intelligent DevOps for Cloud-Native Excellence

The AI-powered CI/CD failure analysis system successfully addresses critical DevOps challenges through intelligent automation. By integrating **Jenkins**, **Docker**, **Kubernetes**, **AWS services**, and **AI-assisted diagnostics**, the solution establishes a structured framework for improving pipeline reliability and operational stability.

The system reduces MTTR, minimizes deployment failures, and provides a practical foundation for intelligent DevOps operations—aligned with cloud-native best practices and industry-relevant methodologies.

Automated
Observability

AI-Powered
Diagnostics

Controlled
Remediation

Reduced
MTTR